

18. Lineáris egyenletek iterációs módszerei a MATLAB-ban

A MATLAB-ban az iterációs módszerek néhány soros, vektorizált kóddal megvalósíthatók, a konjugált gradienshez pedig kész, beépített függvény is van. A tömör mátrix–vektor írásmód (pl. egy sor főátlón kívüli elemeinek indexelése) miatt a kód rövid és gyors marad.

Jacobi iteráció

Az $AX = B$ rendszert oldja meg a P kezdővektorból, szigorúan diagonálisan domináns A esetén konvergálva. Minden komponenst kizárólag az **előző** iteráció értékeiből frissít:

matlab

Kód másolása

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

D = diag(diag(A)); % Diagonális mátrix
L_plus_U = A - D; % Felső és alsó háromszög elemek
x = x0;

for k = 1:maxIter
    x_new = D \ (b - L_plus_U * x); % Jacobi iterációs képlet
    if norm(x_new - x, inf) < tol % Konvergencia ellenőrzése
        break;
    end
    x = x_new;
end

disp('Megoldás:');
disp(x);
```

jacobi(A,B,P,delta,max1) — kézi implementáció

A delta a kívánt tűrés, a max1 az iterációk maximális száma. Érdekes megfigyelni a gépi pontosság (eps) használatát a relatív hiba számításánál, és hogy az $A(j,[1:j-1,j+1:N])$ a j -edik sor főátlón kívüli elemeit adja.

Gauss–Seidel iteráció

Szerkezete csaknem azonos a Jacobiéval — csak a belső ciklus magjában tér el: a már kiszámított $X(1:j-1)$ új értékeket azonnal felhasználja, ezért rendszerint gyorsabban konvergál:

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

L_plus_D = tril(A); % Alsó háromszög és diagonális mátrix
U = A - L_plus_D; % Felső háromszög elemek
x = x0;

for k = 1:maxIter
    x_new = L_plus_D \ (b - U * x); % Gauss-Seidel iterációs képlet
    if norm(x_new - x, inf) < tol % Konvergencia ellenőrzése
        break;
    end
    x = x_new;
end

disp('Megoldás:');
disp(x);
```

gseidel(A,B,P,delta,max1) — kézi implementáció

Konjugált gradiens módszer

A MATLAB-nak van beépített, előkondicionált konjugált gradiens függvénye, a **pcg** (Preconditioned Conjugate Gradient), amely szimmetrikus, pozitív definit A -ra való:

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x0 = [0; 0]; % Kezdeti közelítés
tol = 1e-6; % Konvergencia feltétel
maxIter = 100; % Maximális iterációk száma

% Konjugált gradiens módszer
[x, flag, relres, iter] = pcg(A, b, tol, maxIter, [], [], x0);

disp('Megoldás:');
disp(x);
disp(['Iterációk száma: ', num2str(iter)]);
```

A módszer kézzel is néhány sorban megírható (a lépéshossz, a reziduum és a konjugált irány frissítésével):

matlab

Kód másolása

```
% Mátrix és vektor definiálása
A = [4, 1; 1, 3];
b = [15; 10];
x = [0; 0]; % Kezdeti közelítés
r = b - A * x; % Kezdeti maradék
p = r;
tol = 1e-6; % Konvergencia feltétel

for k = 1:100
    alpha = (r' * r) / (p' * A * p);
    x_new = x + alpha * p;
    r_new = r - alpha * A * p;
    if norm(r_new) < tol
        break;
    end
    beta = (r_new' * r_new) / (r' * r);
    p = r_new + beta * p;
    x = x_new;
    r = r_new;
end

disp('Megoldás:');
disp(x);
```

Manuális CG-implementáció

A módszerek összehasonlítása

Módszer	Sebesség	Alkalmazás
Jacobi	Lassúbb	Ritka mátrixokhoz, egyszerű implementáció
Gauss-Seidel	Gyorsabb, mint Jacobi	Nagyobb mátrixok esetén jobban teljesít
Konjugált gradiens	Leggyorsabb	Szimmetrikus, pozitív definit mátrixokhoz ideális

A módszerek összehasonlítása

A módszer megválasztása a feladattól függ: a Jacobi jól párhuzamosítható, a Gauss–Seidel gyorsabban konvergál, a (pre)konjugált gradiens pedig nagy, ritka, szimmetrikus pozitív definit rendszerekre a leghatékonyabb — tipikusan mérnöki szimulációkban (áramlástan, mechanika, végelelem-analízis) és kvadratikus optimalizálásban.